# Benchmarking Four Artificial-Intelligence Paradigms in Xiangqi: A Controlled and Reproducible Comparison

Alma Nurul Salma (诺艾玛) · Student ID L25020007 · Nanjing Xiaozhuang University (南京晓庄学院)

Supervisory consultation report and research plan · 5 November 2026 · Thesis defense: end of May 2027

***Abstract*—Computer game playing has produced four fundamentally different and historically successful families of artificial intelligence: adversarial tree search, supervised imitation of human experts, self-play reinforcement learning, and the hybrid of Monte Carlo tree search with a learned neural network. Each family has been demonstrated separately and on different games, which makes the published results difficult to compare: the domain, the network architecture, the amount of data and the amount of computation all differ simultaneously. This report proposes and documents a controlled study that removes those confounds. All four paradigms are implemented on a single domain, Xiangqi (Chinese chess), using one shared board representation, one shared action space, one shared neural architecture and one shared evaluation league, so that the only variable that changes is the learning algorithm itself. The report describes the completed system, the three datasets and their processing, the experimental protocol, the results obtained to date, the computational limits of a single-laptop research setting, and a schedule to the thesis defense at the end of May 2027. It closes with the specific questions on which the author seeks the supervisor's guidance.**

***Index Terms*—Xiangqi, Chinese chess, game artificial intelligence, Monte Carlo tree search, imitation learning, reinforcement learning, benchmarking, reproducibility**

## I. INTRODUCTION

### *A. Background*

The history of artificial intelligence in board games is, in effect, a history of four competing answers to a single question: how should a machine decide what to do next? The first answer was explicit search. Shannon formalised the idea that a machine could play chess by enumerating continuations and evaluating the resulting positions with a hand-written function [1], and Knuth and Moore later showed that large parts of that enumeration can be skipped without changing the result, which made the approach practical [2]. This line of work culminated in Deep Blue, the system that defeated the reigning world chess champion using highly optimised search and a human-engineered evaluation function [3].

The second answer was to learn from people. Rather than specifying what a good move is, a network is trained to reproduce the moves that strong human players actually chose. The technique originates in behaviour cloning for autonomous driving, where a network was trained to imitate a human driver's steering [4], and it is now a standard first stage in game-playing systems; a survey of the field describes both its appeal and its central weakness, namely that a cloned policy degrades once it reaches states the demonstrator never visited [5]. That weakness was characterised formally as a compounding-error problem caused by the mismatch between the training distribution and the distribution the learned policy itself induces [6].

The third answer was to discard human data and let the system discover strategy by trial and error. Modern policy-gradient methods made this practical at scale, with Proximal Policy Optimization (PPO) becoming the default algorithm because it is comparatively stable and simple to tune [7]. Self-play reinforcement learning of this kind reached professional human level in domains far less structured than board games, including real-time strategy [8].

The fourth answer combines the previous two. AlphaGo paired a network trained on human games with Monte Carlo tree search and defeated a professional Go player [9]. AlphaGo Zero then removed the human data entirely, learning from self-play alone [10], and AlphaZero generalised the same algorithm to chess and shogi [11]. The search component of these systems derives from Monte Carlo tree search as introduced by Coulom [12] and given its bandit-based selection rule by Kocsis and Szepesvári [13], with the broader design space catalogued in a survey in this journal family [14].

### *B. The problem, and where it comes from*

The four paradigms above are rarely compared against one another. They are compared against *humans*, or against the previous state of the art in whatever domain the authors chose. This matters because the published demonstrations differ along several axes simultaneously. AlphaZero is reported on chess, shogi and Go with a large residual network and thousands of accelerators [11]; PPO is reported on continuous-control benchmarks and video games [7]; behaviour cloning is reported on driving and robotics [5]. A practitioner who asks the straightforward engineering question — *given one game and a fixed budget, which of these four should I build?* — cannot answer it from the literature, because no two of those results share a domain, an architecture, or a compute budget.

This is not a hypothetical concern. It is a documented methodological problem in reinforcement learning: Henderson *et al.* showed that reported performance differences in deep reinforcement learning are frequently attributable to implementation details, random seeds, and evaluation choices rather than to the algorithms being compared, and they argued explicitly for controlled, shared-codebase comparisons with reported variance [15]. The present project is a direct response to that argument,

applied to a domain in which it has not yet been done.

### C. Contributions

This work makes four contributions.

1. A single reproducible codebase in which four distinct AI paradigms play the same game through the same interfaces, so that differences in measured strength are attributable to the algorithms rather than to the surrounding engineering.
2. An evaluation league for Xiangqi that reports relative Elo with confidence intervals, anchored where possible to an open-source engine, together with a held-out move-prediction accuracy metric.
3. An empirical finding, obtained from six separate studies, that at the scale available to a single-laptop researcher the binding constraint on strength is model capacity and training budget rather than the choice of algorithm.
4. An open artefact — a rules engine, three documented datasets, trained checkpoints, and an interactive demonstrator — intended for reuse by subsequent students.

### D. Structure of this report

Section II introduces Xiangqi for readers who do not play it, and explains why the game was selected. Section III reviews the literature and states the gap. Section IV gives the research questions, aim and objectives. Section V presents the solution blueprint and defines each of the four paradigms. Section VI documents the three datasets in full. Section VII gives the experimental design, Section VIII the results obtained so far, and Section IX the computational constraints. Section X discusses the planned output, which is the subject on which the author most wants advice. Section XI gives the schedule, Section XII the expected contributions, Section XIII the risks, and Section XIV the questions put to the supervisor.

## II. THE DOMAIN: XIANGQI

### A. Board and objective

Xiangqi, also called Chinese chess, is played on a grid of nine files and ten ranks. Unlike international chess, the pieces occupy the *intersections* of the lines rather than the squares between them, giving ninety playable points. The two sides are Red, which moves first, and Black. The objective is the same as in chess: to checkmate the opposing General. A side that has no legal move loses, so stalemate is a loss rather than a draw.

Two regions of the board carry special rules. A horizontal band across the middle, the *river*, divides the two halves and changes how pieces behave when they cross it. At each end sits a three-by-three *palace*, marked by diagonals, which the General and the two Advisors may never leave. Fig. 1 shows the starting position with these regions highlighted.

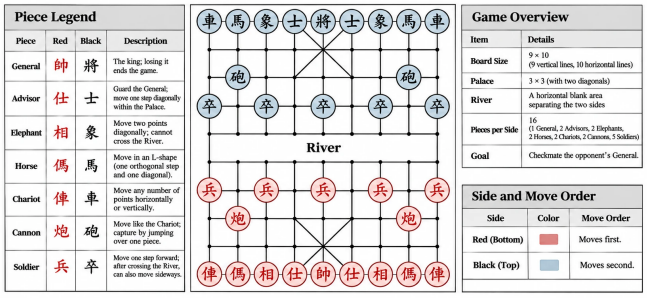



Fig. 1. The Xiangqi board: a 9 x 10 grid of intersections with the river across the middle and a palace at each end, shown in the opening position. Red moves first.

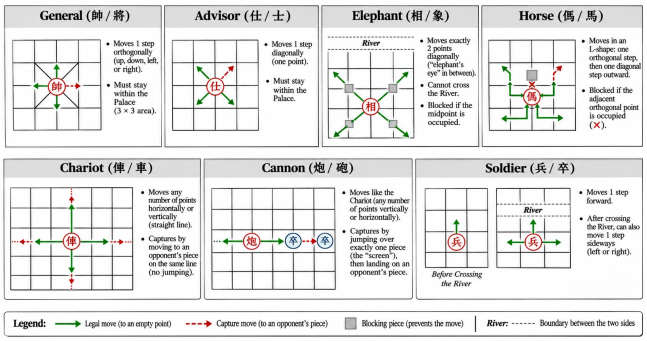



Fig. 2. How each of the seven piece types moves and captures. The cannon jump-the-screen capture and the blockable horse leg have no chess analogue and enlarge the branching factor the agents must search.

### B. The pieces

There are seven piece types per side. Several correspond closely to chess pieces, which makes the game quick to learn: the chariot is a rook, the horse is a knight, and the soldier is a restricted pawn. Three have no chess equivalent and are the source of most of the tactical difficulty. The general and its two advisors never leave the palace; the elephant moves two points diagonally, is blocked by an occupied midpoint, and may never cross the river; and the cannon moves like a chariot but captures only by jumping exactly one intervening piece. Fig. 2 gives the movement rule for every type.

### C. Why the rules make the problem harder

Three rules in particular raise the computational difficulty relative to chess. The Cannon captures only by jumping over exactly one intervening piece, so the legality and value of a capture depends on the contents of the squares between attacker and target, and that relationship changes with every move anywhere on the file or rank. The Horse is blocked by an adjacent piece, so unlike the chess knight it cannot be treated as a fixed offset table. The Soldier gains a lateral step only after crossing the river, so the same piece type has two distinct movement rules depending on where it stands.

The quantitative consequence is that Xiangqi has a larger branching factor and longer games than chess, and therefore a substantially larger game tree; reported figures place the

branching factor near thirty-eight against roughly thirty-five for chess, average game length near ninety-five plies against roughly seventy, and game-tree complexity near $10^{150}$ against roughly $10^{123}$ [16]. Table I summarises the comparison.

TABLE I
XIANGQI COMPARED WITH INTERNATIONAL CHESS

| Property | Xiangqi | International chess |
|---|---|---|
| Board | 9 x 10 intersections (90 points) | 8 x 8 squares (64) |
| Piece types per side | 7 | 6 |
| Average branching factor | about 38 | about 35 |
| Average game length | about 95 plies | about 70 plies |
| Game-tree complexity | about $10^{150}$ | about $10^{123}$ |
| Stalemate | Counts as a loss | Counts as a draw |
| Promotion | None | Pawn promotion |

**Design decision 1 : why Xiangqi rather than chess or Go.** Three reasons. First, *scientific*: the game is demonstrably harder to search than chess [16] while remaining a perfect-information, zero-sum, two-player game, so the classical theory applies without modification and the comparison stays clean. Second, *gap-related*: chess and Go are saturated benchmarks in which a new comparison adds little, whereas Xiangqi has very few published AI studies and none that compare paradigms (Section III). Third, *practical*: a large corpus of recorded human games is publicly available, and a strong open-source engine exists to serve as an absolute reference, which together make a rigorous evaluation possible without proprietary resources.

## III. RELATED WORK AND THE RESEARCH GAP

The author has read the works below, including the Xiangqi-specific papers recommended by the supervisor. Each is summarised for the point it contributes to this project.

### *A. Work on the individual paradigms*

**Adversarial search.** Shannon established the minimax formulation for games [1]; Knuth and Moore analysed alpha-beta pruning and quantified the savings it provides [2]; Campbell *et al.* documented Deep Blue, showing that search plus a hand-engineered evaluation was sufficient to beat the human world champion [3]. *Useful point for this project:* this paradigm needs no data and no training, so it provides a learning-free control condition against which every learned agent can be measured.

**Imitation learning.** Pomerleau demonstrated behaviour cloning by training a network to copy a human driver [4]; Hussein *et al.* survey the methods and their failure modes [5]; Ross *et al.* formalised the distribution-shift problem and proposed an iterative correction [6]. *Useful point:* imitation is cheap and strong in states resembling the training data, but theory predicts degradation in unfamiliar states — a prediction that the endgame results in Section VIII appear to confirm.

**Self-play reinforcement learning.** Sutton and Barto give the formal framework [17]; Schulman *et al.* introduced PPO, the clipped-objective policy-gradient method used here [7]; Vinyals *et al.* showed self-play reaching grandmaster level in a far less structured domain [8]. *Useful point:* self-play needs no human data but is sample-hungry, which makes it the paradigm most sensitive to a limited compute budget.

**Monte Carlo tree search with learned evaluation.** Coulom introduced Monte Carlo tree search [12]; Kocsis and Szepesvári supplied the bandit-based selection rule that makes it converge [13]; Browne *et al.* survey the variants [14]. Silver *et al.* combined the search with deep networks in AlphaGo [9], removed the human data in AlphaGo Zero [10], and generalised the method across games in AlphaZero [11]; Schrittwieser *et al.* extended it to a learned model of the environment [21]. *Useful point:* this is the strongest known recipe, but the published versions were trained with thousands of accelerators [11], which is precisely the condition this project cannot reproduce and therefore must measure honestly.

**Methodology.** Henderson *et al.* demonstrated that reported differences between deep reinforcement learning algorithms are often artefacts of implementation and evaluation choices, and recommended controlled comparisons in a shared codebase with reported variance [15]. *Useful point:* this paper supplies the methodological justification for the entire design of this project. Tian *et al.* reimplemented AlphaZero openly and reported the resources required, providing a concrete reference for the compute gap discussed in Section IX [18].

### *B. Work specific to Xiangqi*

Li *et al.* introduced JiangJun, a Xiangqi system that addresses non-transitivity in two-player zero-sum games; it is trained on a large corpus of human games and is, to the author's knowledge, the most substantial peer-reviewed Xiangqi learning system to date [19]. *Useful point:* it establishes Xiangqi as a venue for serious machine-learning research and provides a reference for corpus-scale supervised training, but it studies a single method rather than comparing families.

Yilmaz *et al.* describe an AlphaZero-style Xiangqi player combining deep reinforcement learning with Monte Carlo tree search, and report the complexity figures used in Table II [16]. *Useful point:* it confirms that the AlphaZero recipe transfers to Xiangqi, and supplies the domain-complexity numbers; again, it evaluates one method.

Chen *et al.* propose Xiangqi-R1, using Xiangqi as a testbed for spatial and strategic reasoning in large language models trained with reinforcement learning [20]. *Useful point:* it shows the research community now treats Xiangqi as a reasoning benchmark rather than a curiosity, which strengthens the motivation; it is nonetheless orthogonal to the classical paradigms compared here.

### *C. The gap*

Table II makes the gap explicit. Every study listed advances one paradigm. None holds the domain, the representation, the architecture and the evaluation fixed while varying the paradigm, and none reports a like-for-like ranking

of the four families on Xiangqi together with an analysis of *why* the ranking comes out as it does.

TABLE II
POSITIONING OF THIS PROJECT AGAINST THE LITERATURE

| Study | Domain | Paradigms studied | Compares paradigms head to head? | Analyses why one wins? |
|---|---|---|---|---|
| Campbell et al. [3] | Chess | Search | No | No |
| Silver et al. [9], [10], [11] | Go, chess, shogi | MCTS + network (and IL as a stage) | Partly, as ablations within one system | Partly |
| Schulman et al. [7] | Control, video games | Self-play RL | No | No |
| Li et al. [19] | Xiangqi | Supervised + population methods | No | No |
| Yilmaz et al. [16] | Xiangqi | MCTS + network | No | No |
| Chen et al. [20] | Xiangqi | LLM reasoning | No | No |
| **This project** | **Xiangqi** | **All four, shared code** | **Yes** | **Yes, that is the objective** |

**Statement of the problem.** *There is no controlled, reproducible comparison of the four principal game-AI paradigms on a single domain, and consequently no evidence-based guidance on which paradigm to choose, or on what actually limits each one, when resources are constrained.*

**Statement of the solution.** *Implement all four paradigms over one shared Xiangqi environment, one shared state and action representation, and one shared network; evaluate them in a single Elo league with confidence intervals and an engine anchor; and attribute the resulting differences through targeted ablations on search, capacity, data source and training budget.*

## IV. RESEARCH QUESTIONS, AIM AND OBJECTIVES

### *A. Aim*

To produce a controlled, reproducible and analysed answer to the question of which artificial-intelligence paradigm plays Xiangqi best under a fixed and honestly reported budget, and to deliver the evaluation league as a reusable research artefact.

### *B. Research questions*

**RQ1 — Ranking.** When the domain, the state representation, the action space, the network architecture and the evaluation protocol are held constant, how do adversarial search, imitation learning, self-play reinforcement learning, and Monte Carlo tree search with a learned network rank against one another on a single Elo scale?

**RQ2 — The imitation effect.** Does initialising self-play reinforcement learning from an imitation-learned policy improve the final agent relative to learning from random initialisation, or does it constrain it? The literature motivates both outcomes: imitation gives a strong starting policy [5], but it also imposes the demonstrator's distribution, which can be a liability once the policy departs from it [6].

**RQ3 — The binding constraint.** When an agent stops improving, which factor is responsible: the depth of search, the capacity of the network, the quality or quantity of the training data, or the learning algorithm itself?

### *C. Objectives*

- **O1.** Implement a complete, rules-correct Xiangqi environment with a fixed action space and a tensor encoding shared by all learned agents.
- **O2.** Implement the four paradigms over that shared substrate, plus a random floor and an engine ceiling.
- **O3.** Build an evaluation league reporting relative Elo with 95% confidence intervals, together with a held-out move-prediction metric.
- **O4.** Run targeted ablations that isolate search budget, network capacity, and data source, in order to answer RQ3.
- **O5.** Release the environment, the datasets, the checkpoints and an interactive demonstrator so that the comparison can be rerun and extended.

## V. SOLUTION BLUEPRINT

### *A. System overview*

The system is organised as a single substrate shared by every agent. A rules engine produces legal moves; an encoder converts any position into a fixed tensor; a fixed action space assigns every geometrically possible move an integer index; a legality mask prevents any agent from selecting an illegal action; and a tournament harness plays agents against each other and computes ratings. Only the decision procedure between "encoded position" and "chosen action" differs between paradigms. This is the structural device that makes the comparison controlled, and it follows directly from the methodological recommendation of Henderson *et al.* [15].

Fig. 3 shows the apparatus as a whole: one shared engine, action space and policy-value network feed four interchangeable decision functions, and every agent is rated on a single Elo scale by a colour-balanced round robin anchored to an engine.

Concretely, the substrate is five modules. The *environment* implements the rules and generates legal moves. The *action space* assigns every geometrically possible move a fixed index. The *encoder* turns a position into a tensor. The *network* maps that tensor to a move distribution and a scalar evaluation. The *arena* plays agents against each other and fits ratings. Every agent implements a single interface, `select_move(board) -> action`, and nothing else about an agent is visible to the rest of the system. Adding a fifth paradigm therefore requires implementing one function, which is what makes the league reusable rather than a one-off experiment.

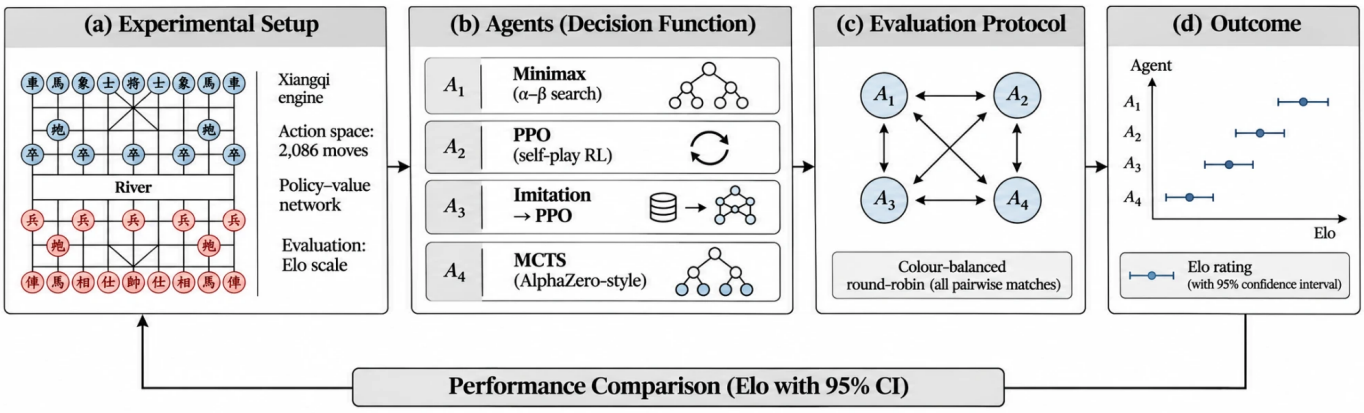



Fig. 3. The controlled apparatus. One shared engine, action space and policy-value network feed four interchangeable decision functions, which are the only variable, and all agents are rated on a single Elo scale by a colour-balanced round robin anchored to Pikafish.

### *B. State representation, action space and masking*

A position is encoded as a binary tensor of shape 14 x 10 x 9. The ten-by-nine plane matches the board; the fourteen channels are the seven piece types of the side to move followed by the seven piece types of the opponent. Encoding from the perspective of the side to move means the network always "plays Red", which removes the need for a separate side-to-move input and halves the symmetry the network must learn.

The action space is a fixed list of 2,086 entries covering every geometrically possible move on the board. In any given position only a small subset is legal — about forty-four in the opening. The environment therefore produces a boolean legality mask of length 2,086, and every agent applies it before selecting: illegal logits are set to negative infinity so that their probability after the softmax is exactly zero. The same mask is supplied to the reinforcement-learning trainer, so illegal actions are excluded during learning as well as during play. Fig. 4 illustrates the encoding. Formally, with raw logits $z$ and a binary legality mask $m$,

$$\text{mask}|z'_i = z_i \text{ if } m_i = 1, \quad z'_i = -\infty \text{ otherwise}; \quad \mathbf{p} = \text{softmax}(z') \tag{1}$$

so the probability assigned to any illegal move is exactly zero rather than merely small. This guarantees that no agent can ever propose an illegal move, that no gradient is spent on unplayable actions during training, and that all learned agents face an identical decision problem.

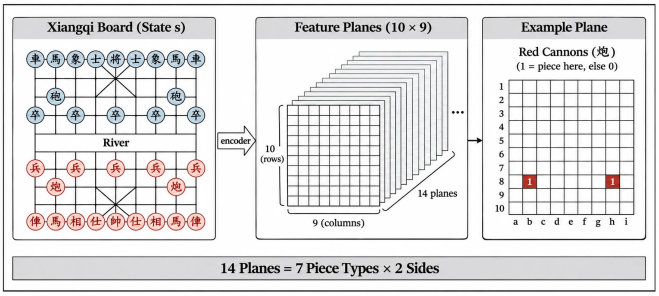
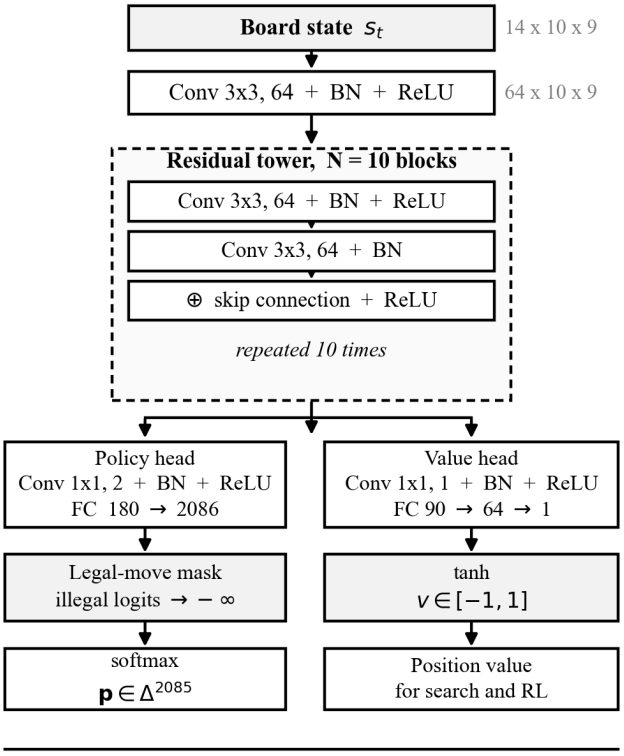



Fig. 4. The board is encoded as a 14 x 10 x 9 tensor: one 10 x 9 binary plane per piece type and side, always from the perspective of the side to move, and shared by every agent.

### *C. The shared network*

Imitation learning, self-play reinforcement learning and Monte Carlo tree search all use the same policy-value network, illustrated in Fig. 5: an input convolution, a tower of ten residual blocks of width sixty-four, and two heads — a policy head emitting 2,086 logits and a value head emitting a scalar in the interval from minus one to one. The network has approximately 1.15 million parameters. Using one architecture for three paradigms is deliberate: it means that any measured difference between them cannot be attributed to the network.




Total 1.15 M parameters. The identical network is shared by M2, M3 and M4, so the comparison isolates the learning rule.

Fig. 5. The shared policy-value network. The same architecture serves imitation learning, PPO self-play and MCTS, so the comparison isolates the learning algorithm.

**Design decision 2 : why one shared network.** If each paradigm were given the architecture that suits it best, the study would measure "best engineering effort per paradigm", which is unreproducible and depends on the author's familiarity with each method. Holding the architecture fixed sacrifices some peak performance in exchange for an interpretable comparison, which is the stated aim.

*D. Method M1 — Minimax with alpha-beta pruning*

*Definition.* The agent treats the game as a finite zero-sum tree and assumes optimal play by both sides. It expands the tree to a fixed depth, evaluates the leaves with a hand-written function, and propagates values upward by alternating maximisation and minimisation [1]. Alpha-beta pruning discards branches that provably cannot affect the root value [2].

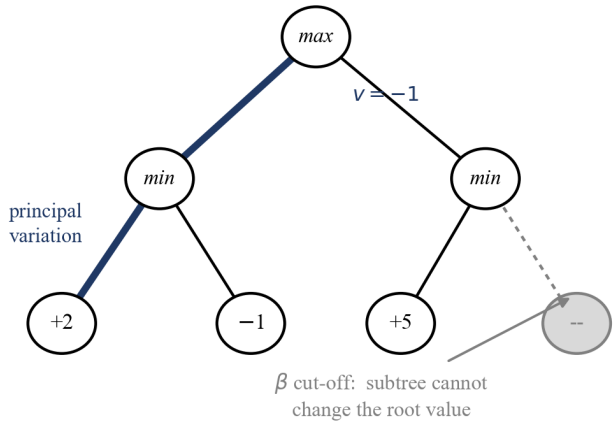
*Flow (Fig. 6).* From the current position, generate legal moves; recurse to the configured depth, alternating between maximising and minimising layers; score each leaf by material balance; prune any subtree bounded out by the running alpha and beta values; return the move on the principal variation.

Writing $T(s, a)$ for the position reached by playing $a$ in $s$, and $h$ for the hand-written evaluation, the value backed up to the root is

$$\text{minimax}|V_d(s) = \max_{a \in A(s)} [\, -V_{d-1}(T(s,a)) \,], \quad V_0(s) = h(s) \tag{2}$$

with alpha-beta pruning discarding any subtree whose bounds prove it cannot alter the value at the root, which leaves the result unchanged while removing a large fraction of the work [2].

*Configuration.* Depths of one, two and three plies are used. Depth two is the standard opponent in the league. There is no training and no data; the only tunable quantity is depth.




Evaluation $h(s) = \sum_i w_i (n_i^R - n_i^B)$, with $w = (9, 4.5, 2, 2, 1, 1)$ for chariot, cannon, horse, elephant, advisor and soldier; depth $d \in \{1, 2, 3\}$. No parameters are learned, so strength is a pure function of $d$ and the agent serves as the learning-free control condition for the whole study.

Fig. 6. Method M1. Values alternate between maximising and minimising layers; the dotted branch is removed by alpha-beta pruning; the thick path is the selected line.

**Design decision 3 : why include a method that cannot learn.** M1 is the control condition. Without it, a result such as "the learned agent reaches 1600 Elo" has no meaning. With it, every learned agent can be expressed as a gain or a loss relative to a system that uses no data at all.

*E. Method M2 — Imitation learning*

*Definition.* Supervised learning of a policy from recorded expert decisions. Each position from a human game becomes a training example whose policy target is the move the human played and whose value target is the eventual result of that game [4], [5].

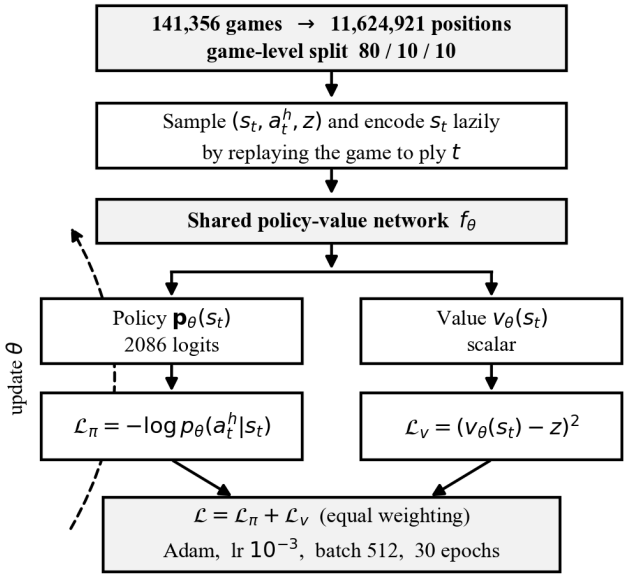
*Flow (Fig. 7).* Sample a position from the corpus; encode it as a 14 x 10 x 9 tensor; pass it through the shared network; compute cross-entropy between the predicted move distribution and the human's actual move, and mean-squared error between the predicted value and the game outcome; update by gradient descent. Repeat for thirty epochs over 9.3 million training positions.

For a sampled position $s$ with the human's move $a$<super>h</super> and the eventual game result $z$ taken from the mover's perspective, the objective is the sum of a classification term and a regression term,

$$\text{ilos}|L(\theta) = -\log p_\theta(a^{\text{h}}|s) + (\, v_\theta(s) - z\,)^2 \qquad (3)$$

weighted equally. No search is performed at decision time: the agent simply plays the highest-probability legal move.

*Configuration.* Learning rate 1e-3, batch size 512, Adam optimiser, thirty epochs. Because 11.6 million encoded tensors do not fit in memory, positions are encoded lazily by replaying each game on access.




The target is the move a human master actually played, so the agent reproduces human play but cannot exceed the quality of the demonstrations. Theory predicts degradation away from the demonstrated state distribution; Section VIII-C reports exactly that pattern across the three phases of the game.

Fig. 7. Method M2. The training signal is the move a human expert actually played, so the agent inherits human strategy but is bounded by the quality of the demonstrations.

### *F. Method M3 — Self-play reinforcement learning with PPO*

*Definition.* The agent improves by playing against a copy of itself and adjusting its policy in the direction of outcomes that produced wins, using the clipped surrogate objective of PPO to limit the size of each update [7], [17].

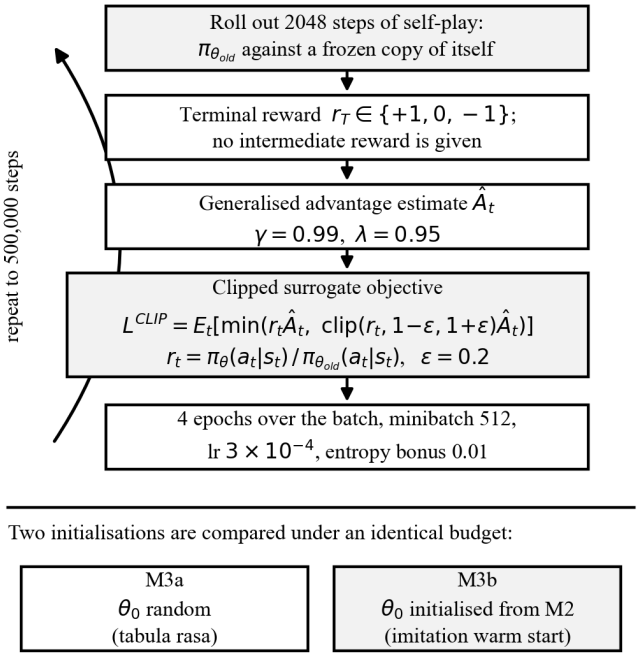
*Flow (Fig. 8).* Play a complete game against a copy of the current policy with illegal actions masked; assign a terminal reward of plus one for a win, minus one for a loss and zero for a draw; compute advantages; apply the clipped PPO update; repeat.

With the probability ratio *r*<sub>t</sub> between the new and the old policy and an advantage estimate Â<sub>t</sub>, the update maximises the clipped surrogate

$$\text{ppo}|L^{\text{CLIP}}(\theta) = \mathrm{E}_t\,[\,\min(\, r_t\,\hat{\mathrm{A}}_t,\ \text{clip}(r_t,\, 1-\varepsilon,\, 1+\varepsilon)\,\hat{\mathrm{A}}_t\,)\,] \qquad (4)$$

whose clipping term is what keeps each update from moving the policy so far that the collected data stop being informative [7].

*Configuration.* Learning rate 3e-4, 2,048 steps per update, batch size 512, four epochs per update, clipping parameter 0.2, discount 0.99, entropy coefficient 0.01, 500,000 environment steps. Two variants are trained: **M3a** from random initialisation, and **M3b** initialised from the M2 network. The contrast between them is the experiment that answers RQ2.




The contrast between M3a and M3b is the direct test of RQ2: whether imitation pretraining helps or constrains the self-play that follows. Illegal actions are masked inside the trainer, so no gradient is ever spent on an unplayable move.

Fig. 8. Method M3. The two initialisations, random and imitation-pretrained, make this method a direct test of RQ2.

### *G. Method M4 — Monte Carlo tree search with the learned network*

*Definition.* Before each move the agent builds a search tree by simulation. The network supplies prior probabilities and a position evaluation, which replaces the random rollouts of classical MCTS [12], [13], [14]; the construction follows AlphaZero [11].

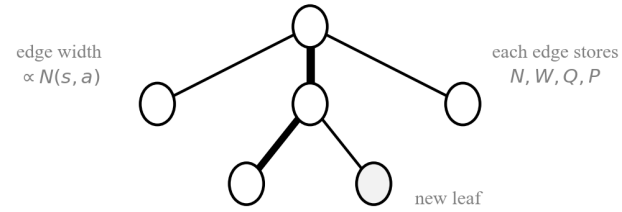
*Flow (Fig. 9).* **Select** — descend the tree using the PUCT rule, which balances the accumulated action value against the network's prior scaled by an exploration term. **Expand** — add the leaf reached and enumerate its legal moves. **Evaluate** — query the network for the leaf's priors and value. **Back up** — propagate the value along the traversed path, incrementing visit counts. After the simulation budget is exhausted, play the most-visited move.

Each edge of the tree stores a visit count $N$, a total value $W$, a mean value $Q = W/N$, and the network prior $P$. Selection descends by the PUCT rule

$$\text{puct}|a^* = \arg\max_a \left[\, Q(s,a) + c_{\text{puct}}\, P(s,a)\, \sqrt{\left(\, \Sigma_b\, N(s,b)\,\right)} \,/\, \left(1 + N(s,a)\,\right)\,\right] \qquad (5)$$

which trades the value already measured against the prior weighted by how little the edge has been explored [13], [14]. Crucially, the leaf is scored by a single network evaluation rather than by a random rollout, which is the substitution that made the method effective in AlphaGo [9].

*Configuration.* Exploration constant 1.5; one hundred simulations per move during training and forty-eight in the live demonstrator; Dirichlet noise with alpha 0.3 and weight 0.25 at the root during self-play; visit-count sampling for the first thirty plies to maintain diversity. Training runs for twenty iterations of twenty-five self-play games, with two network epochs per iteration, warm-started from the M2 checkpoint.




**1) Select**
descend by $a^* = \arg\max_a [\, Q(s,a) + c_{puct} P(s,a) \frac{\sqrt{\sum_b N(s,b)}}{1+N(s,a)} ]$, $c_{puct} = 1.5$

**2) Expand**
add the leaf $s_L$ and enumerate its legal moves

**3) Evaluate**
one network pass: $(\mathbf{p}, v) = f_\theta(s_L)$ replaces the random rollout

**4) Back up**
for every edge on the path: $N \leftarrow N+1$, $W \leftarrow W+v$, $Q = W/N$

| Training self-play | Move selection |
|---|---|
| $n = 100$ simulations<br>Dirichlet $\alpha = 0.3$, $\varepsilon = 0.25$ | visit sampling for 30 plies,<br>then the most-visited move |

The visit distribution $\pi(a) \propto N(s,a)^{1/\tau}$ is itself the policy target during training, so search quality is distilled back into $f_\theta$. Twenty iterations of twenty-five games, with two network epochs per iteration.

Fig. 9. Method M4. The network supplies intuition and the tree search verifies it; during training the visit distribution itself becomes the policy target.

**Design decision 4 : why these four methods and not others.** The four were selected because they are the four historically distinct answers to the move-selection problem, each with a landmark result behind it: search [3], imitation [4], self-play [8], and the hybrid [11]. Together they span the two axes that matter — whether the agent *searches* at decision time, and where its knowledge *comes from* (hand-written, human, or self-generated). Table III shows that the four fill that two-by-two space, which is why adding a fifth method would not add a new category.

TABLE III
THE FOUR PARADIGMS SPAN TWO DESIGN AXES

| | **Knowledge is hand-written** | **Knowledge from human data** | **Knowledge from self-play** |
|---|---|---|---|
| **No search at decision time** | — | M2 Imitation learning | M3 PPO self-play |
| **Search at decision time** | M1 Minimax | (M4 warm start) | M4 MCTS + network |

### *H. Implementation and reproducibility*

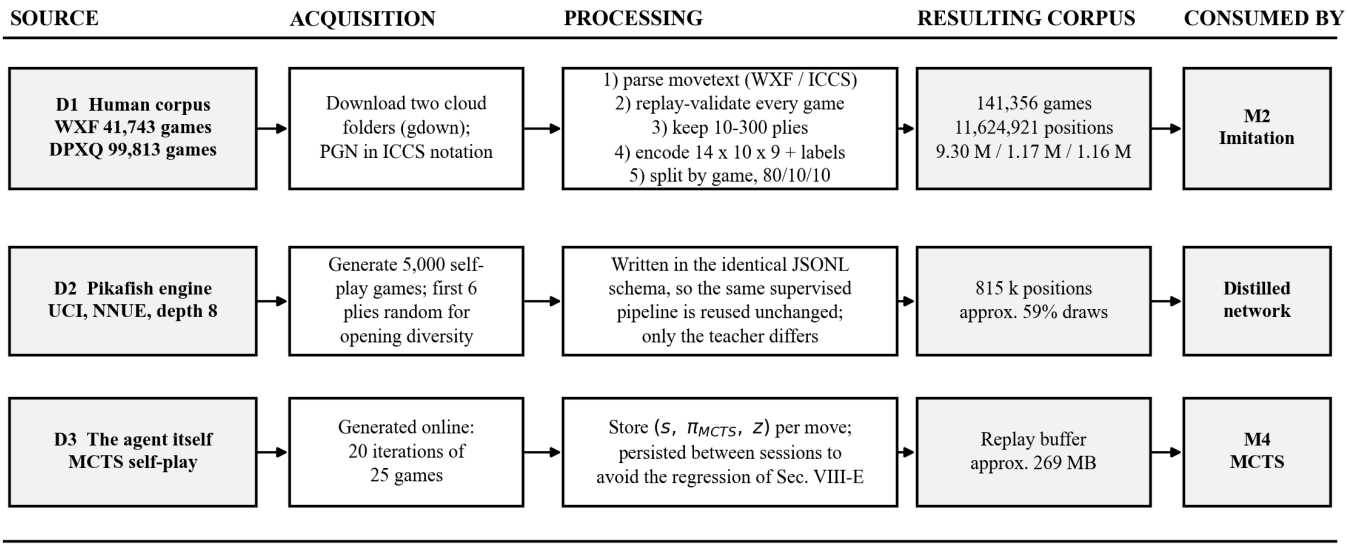
The system is written in Python with PyTorch, and reinforcement learning uses the maskable PPO implementation of the Stable-Baselines3 contributed library. Three properties were designed in deliberately. First, *determinism where it matters*: the dataset split is seeded and its statistics are recorded in a committed manifest, so the corpus can be rebuilt exactly. Second, *interruptibility*: every trainer checkpoints within an epoch as well as between epochs and can resume, because a single laptop cannot be relied upon to run for days uninterrupted; the MCTS trainer additionally persists its replay buffer, for the reason given in Section VIII-E. Third, *testability*: 224 automated tests cover the environment, the data pipeline, the models, the agents, the trainers and the evaluation code, and the replay-validation of 141,356 recorded human games doubles as a large-scale conformance test of the rule engine itself.

### *I. Reference points*

Two non-paradigm agents bound the scale. A **random legal-move player** is the floor; any agent that fails to beat it decisively is not functioning. **Pikafish**, a strong open-source UCI Xiangqi engine, is the ceiling and is also used as a teacher in a distillation study and as the position evaluator in the demonstrator.

## VI. DATA: SOURCES AND PROCESSING

Three distinct datasets are used. One is downloaded; two are generated by the project itself. All three are written in the same JSON-lines schema, which records the game outcome and the move sequence, so that a single loading and encoding path serves all of them. Fig. 10 shows the complete pipeline.




All three corpora share one JSON-lines schema, so a single loader and encoder serves every paradigm. PPO (M3) also generates experience by self-play, but it is consumed immediately by the on-policy update and is therefore not retained as a dataset.

Fig. 10. The full data pipeline for the three datasets, from source to the training stage that consumes each one.

### *A. Dataset D1 — the human game corpus*

*Provenance.* The human corpus is the publicly released collection *chinese-chess-PGN*, which bundles two independently assembled archives: records from the World Xiangqi Federation, and records from the DPXQ archive associated with the Chinese Xiangqi portal dpxq.com. Both are distributed in standard PGN with ICCS coordinate movetext. Fig. 11 reproduces the repository documentation that states the two collection sizes.

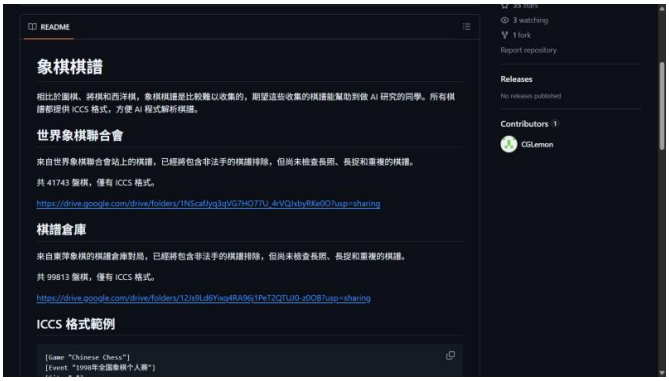



Fig. 11. The documentation of the public repository from which the human corpus was obtained, stating the two archives: 41,743 games from the World Xiangqi Federation and 99,813 from the DPXQ archive, each with its distribution link.

*Acquisition.* The author first attempted to build a web scraper to collect games directly from Xiangqi portals. That attempt was abandoned: the browser-automation driver assets could not be provisioned reliably, and the resulting data would in any case have required the same cleaning as a published corpus while adding legal and reproducibility uncertainty. The project therefore switched to the published collection, whose two archives are distributed as cloud-hosted folders and were downloaded programmatically into the raw-data directory. This is recorded here because provenance should be reportable, and because the change of approach is itself a methodological decision.

**Design decision 5 — a published corpus rather than a self-built scrape.** A published, citable corpus makes the dataset reproducible by a third party, removes scraping-related legal ambiguity, and lets the project spend its limited time on the comparison rather than on data collection. The cost is that the author does not control the sampling; this is mitigated by validating every game against the rules engine, as described below.

*Processing.* The pipeline executes five stages. **(1) Parse** the movetext, auto-detecting WXF or ICCS notation. **(2) Replay-validate** every game move by move through the project's own rules engine, discarding any game containing an illegal or unparseable move; this both cleans the data and acts as a test of the rules engine itself. **(3) Filter** by length, retaining games between ten and three hundred plies. **(4) Encode** each position into the 14 x 10 x 9 tensor with its policy label (the index of the move played) and value label (the game result). **(5) Split** at the level of whole games, eighty/ten/ten with a fixed seed, so that no position from a training game can appear in validation or test.

*Outcome.* Table IV reports the resulting statistics, all of which are recorded in a committed manifest for reproducibility. A random sample of five hundred games was re-validated independently and all five hundred replayed legally.

TABLE IV
DATASET D1 AFTER PROCESSING

| Quantity | Value |
|---|---|
| Games downloaded (WXF + DPXQ) | 41,743 + 99,813 = 141,556 |
| Rejected: illegal | 0 |
| Rejected: shorter than 10 plies | 26 |
| Rejected: longer than 300 plies | 174 |
| Games retained | 141,356 |

| Quantity | Value |
|---|---|
| Positions (training examples) | 11,624,921 |
| Split: train / validation / test (games) | 113,084 / 14,135 / 14,137 |
| Split: train / validation / test (positions) | 9,296,810 / 1,165,000 / 1,163,111 |
| Outcomes: Red win / Black win / draw | 53,764 / 39,860 / 47,732 |
| Mean game length | 82.24 plies |
| Spot-check legality (500 games) | 500 / 500 |

### *B. Dataset D2 — engine self-play for distillation*

*Provenance.* Generated by the project using Pikafish, an open-source UCI Xiangqi engine (Fig. 12), playing against itself.

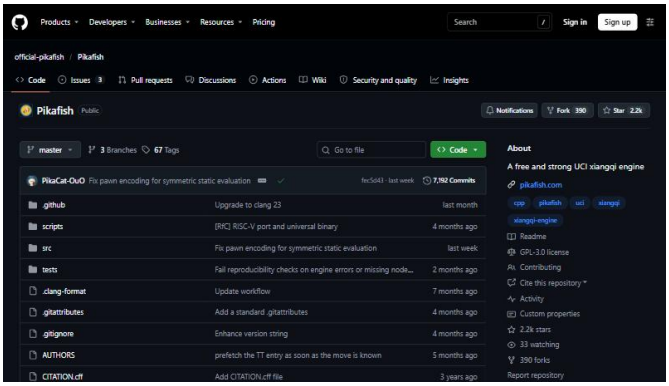



Fig. 12. Pikafish, the open-source engine used as the absolute reference point, as a distillation teacher, and as the position evaluator in the demonstrator.

*Processing.* A deterministic engine playing itself would produce the same game repeatedly, so the first six plies of each game are random legal moves to force opening diversity; thereafter both sides are played by the engine at search depth eight. A game terminates on checkmate, stalemate, position repetition, or the ply limit. Each finished game is appended immediately to the output file, which makes a long generation run crash-safe and resumable. Because the output uses the same schema as D1, the identical supervised training code consumes it without modification; the only thing that changes is the teacher.

*Outcome.* Five thousand games were generated in approximately 2.2 hours, yielding roughly 815,000 positions, of which about fifty-nine per cent were draws.

**Design decision 6 : why distil from an engine at all.** If the limiting factor were the *quality of the teacher*, then replacing human demonstrations with a superhuman engine should produce a stronger student. This dataset therefore exists to test one specific hypothesis within RQ3, not to produce the best possible agent. Section VIII reports that the hypothesis was not supported, which is itself an informative result.

### *C. Dataset D3 — MCTS self-play replay buffer*

*Provenance.* Generated by the agent itself while method M4 trains.

*Processing.* During each self-play game the agent runs the full tree search at every move and records three things: the encoded position, the normalised visit-count distribution over legal moves (which becomes the policy target), and, once the game ends, the final result from that position's perspective (the value target). Examples accumulate in a replay buffer that is persisted to disk between training sessions.

*Outcome.* Twenty iterations of twenty-five games produce a buffer of approximately 269 MB. Persisting this buffer was not a convenience but a correctness fix: an earlier version discarded the buffer whenever training was resumed, and the resulting agent regressed substantially, as reported in Section VIII.

*Note on PPO.* Method M3 also generates experience by self-play, but it is consumed immediately by the on-policy update and is not retained as a dataset, so it is not counted among the three.

## VII. EXPERIMENTAL DESIGN AND EVALUATION PROTOCOL

### *A. Primary metric: relative Elo from a round-robin league*

Agents are evaluated by playing a complete round robin and converting the resulting score matrix into Elo ratings [22]. Ratings are *relative*: the pool is anchored at 1500, so a number is meaningful only against the other agents in the same tournament, and never as an official human rating. Every reported rating carries a 95% confidence interval, following the recommendation that variance be reported rather than suppressed [15]. Colour is alternated so that no agent benefits from moving first. Ratings are fitted so that the expected score between two agents follows the logistic relation

$$\mathrm{elo}|E_{\mathrm{A}} = 1 / ( 1 + 10^{(R_{\mathrm{B}} - R_{\mathrm{A}})/400} ) \qquad (6)$$

and a 95 per cent interval is reported for every rating [22].

### *B. Secondary metric: held-out move-prediction accuracy*

Elo measures playing strength but not what a network has learned. A second metric therefore measures how often the imitation network predicts the move a human master actually played, on the test split, which is disjoint from training at the level of whole games. Both the raw top-one accuracy over all 2,086 actions and the legality-masked top-one and top-three accuracies are reported, and results are broken down by game phase.

### *C. Controls*

Four controls keep the comparison fair: a single rules engine and action space for all agents; a single network architecture for all learned agents; a matched number of self-play steps where two learning methods are compared; and a fixed random seed with the split recorded in a committed manifest.

### *D. The studies*

Six studies have been completed, each targeting a specific question (Table V).

TABLE V
THE SIX COMPLETED STUDIES

| # | Study | Question it answers | Design |
|---|---|---|---|
| S1 | Seven-agent round robin | RQ1, RQ2 | All paradigm variants plus random, 50 games per pair |
| S2 | MCTS entry | RQ1 | M4 added to the league |
| S3 | MCTS stabilised (v2) | Validity | Re-run after the replay-buffer fix |
| S4 | Engine distillation | RQ3 (teacher quality) | Human-taught vs engine-taught network |
| S5 | Capacity ablation | RQ3 (capacity) | 1.15 M vs 4.85 M parameters, same tournament |
| S6 | Search ablation | RQ3 (search) | M4 at 100, 200, 400, 800 simulations |

### *E. Threats to validity*

Three are acknowledged. *Scale*: all conclusions hold at the compute budget of Section IX and may not extrapolate. *Anchoring*: Elo is comparable within a tournament, not across tournaments, so cross-study comparisons in this report are made only where agents appeared in the same league. *Sample size*: the later leagues use 80 to 100 games per agent, which yields confidence intervals of roughly 70 to 100 Elo; differences smaller than that are not claimed as significant.

## VIII. RESULTS OBTAINED SO FAR

### *A. Ranking (RQ1)*

Fig. 13 gives the main league. Monte Carlo tree search over the learned network is the strongest paradigm at 1691 Elo, ahead of imitation learning at 1597, minimax at 1460, imitation-initialised PPO at 1452, and the random floor at 1300. Fig. 14 repeats the exercise with the engine included, which places all learned agents on a scale that has an external reference: Pikafish at search depth eight reaches 2062, roughly 570 Elo above the best project agent in that league.

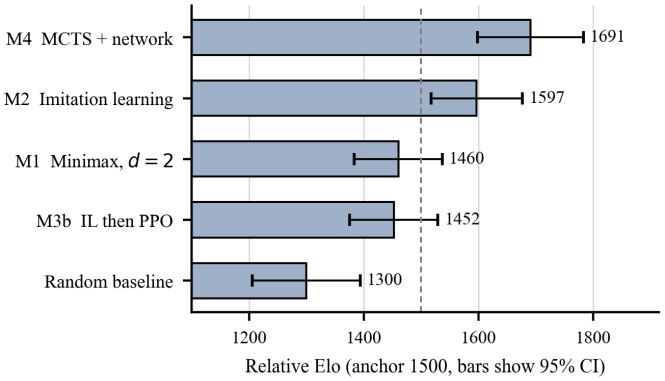



Fig. 13. Main league: relative Elo of the four paradigms with 95 per cent confidence intervals, from a colour-balanced round robin anchored at 1500.

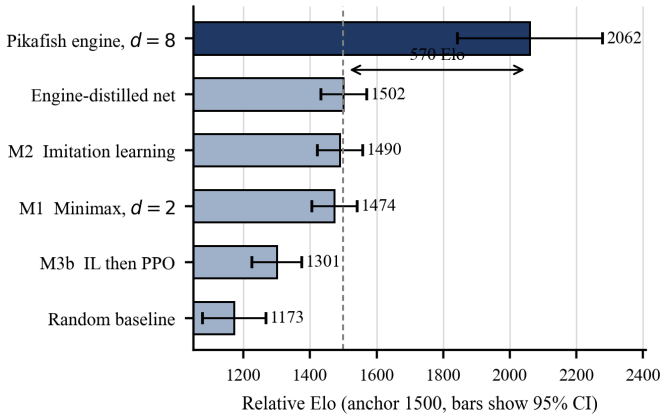



Fig. 14. Engine-anchored league. Including Pikafish at search depth eight places every learned agent on a scale with an external reference and exposes a gap of about 570 Elo.

### *B. The imitation effect (RQ2)*

The result here was contrary to expectation and is the most interesting finding so far. In the seven-agent round robin, imitation learning *alone* scored 1689 Elo, while every self-play variant scored between 1444 and 1526 — including the variants initialised from the imitation network. Self-play at the budget available did not improve on imitation; in the self-play-opponent configuration it actively degraded it. The explanation consistent with the literature is sample efficiency: policy-gradient self-play is known to require very large numbers of environment interactions [7], [8], and 500,000 steps is several orders of magnitude below what the landmark results used [11], [18]. The honest conclusion at this stage is therefore not "self-play is inferior" but "self-play is not competitive at this budget", which is exactly the kind of claim this project is designed to make precisely.

### *C. What the network learned*

Fig. 15 breaks down the imitation network's accuracy by game phase. It predicts the human's exact move 63.2% of the time in the opening but only 32.3% in the endgame. This is the behaviour the imitation-learning literature predicts: the opening is a comparatively small, heavily repeated region of the state space that is well covered by the demonstrations, whereas endgames are long-tailed and tactical, and performance degrades where coverage thins [5], [6].

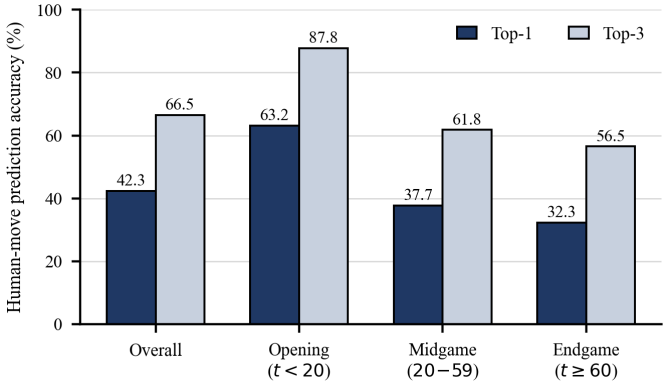



Fig. 15. Move-prediction accuracy of the imitation network on 163,416 held-out positions, by game phase.

### D. The binding constraint (RQ3)

Four separate interventions were attempted to push strength higher, and none produced a significant gain.

1. **More search.** Increasing the MCTS simulation budget eightfold, from 100 to 800 per move, did not improve the score against the network baseline (Fig. 16).
2. **More capacity.** Quadrupling the network from 1.15 M to 4.85 M parameters produced a 43-Elo change within the same tournament, well inside the 95% confidence interval of roughly 78 Elo (Fig. 17).
3. **A stronger teacher.** The engine-distilled network reached 1502 Elo against 1490 for the human-taught network in the same league — statistically indistinguishable — and its move-prediction accuracy plateaued far below that of the human-taught network.
4. **More training.** Additional epochs plateaued rather than continuing to improve.

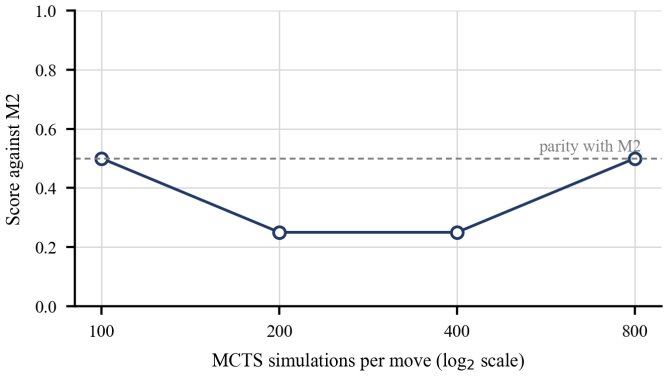



Fig. 16. Search ablation. Increasing the MCTS simulation budget eightfold does not improve the score against the M2 network.

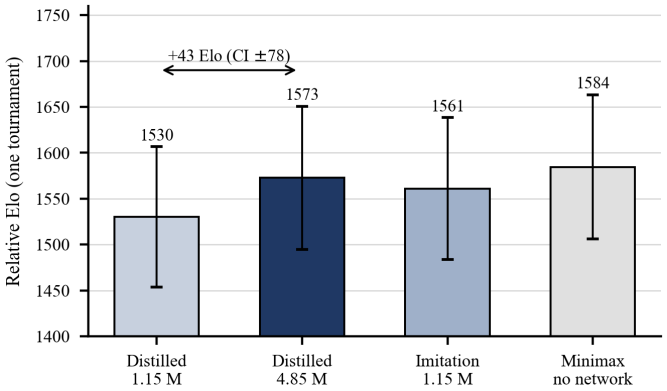



Fig. 17. Capacity ablation, measured inside a single tournament so the ratings are directly comparable. Quadrupling the parameter count moves the rating by less than the confidence interval.

Four independent levers converging on the same ceiling is the signature of a shared upstream constraint rather than four separate algorithmic limits. The interpretation adopted here is that at this scale the system is limited by model capacity in combination with the volume of training signal that the available compute can deliver — a relationship that has been characterised quantitatively in other domains, where model size and data must be scaled together for either to help [23], [24]. It is important to state what this does *not* claim: it does not claim that Xiangqi has an intrinsic 1690 ceiling, only that this budget does.

### E. A methodological finding

During the first MCTS training run the replay buffer was discarded each time training was resumed after an interruption. The agent regressed from 1703 to 1582 Elo. After the buffer was made persistent and a best-checkpoint criterion was added, a retrained agent recovered to 1691. The regression was therefore caused by the training infrastructure, not by the algorithm. This is a small, concrete instance of precisely the phenomenon Henderson *et al.* warned about [15], and it is reported because it would otherwise have been silently attributed to the method.

## IX. COMPUTATIONAL CONSTRAINTS

### A. The available hardware

All training was performed on a single consumer laptop (Table VI). Sustained training caused thermal throttling, which was managed by training in cooled bursts with reduced batch sizes and worker counts.

TABLE VI
TRAINING ENVIRONMENT

| Component | Specification |
|---|---|
| GPU | NVIDIA GeForce RTX 4060 Laptop GPU, 8 GB VRAM |
| Framework | PyTorch 2.14 with CUDA 12.6, Python 3.12 |
| Reinforcement learning | Stable-Baselines3 / sb3-contrib (MaskablePPO) |
| Engine reference | Pikafish (UCI), run on CPU |
| Total training time | On the order of 100 GPU-hours across all runs |
| Largest single run | MCTS self-play, executed across multiple resumed sessions |

### B. The scale gap

Fig. 18 places this budget beside the published landmark systems. AlphaZero used five thousand first-generation tensor processing units to generate self-play games together with sixty-four second-generation units for training [11]; an open reimplementation reported on the order of two thousand GPUs [18]. The gap to this project is three to four orders of magnitude in accelerators, before accounting for duration.

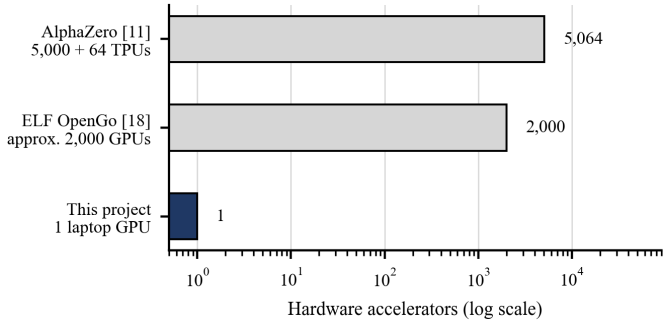



Fig. 18. Accelerators used by landmark systems compared with this project, on a logarithmic scale. Figures are taken from the cited papers.

### C. What this means for the project

Two consequences follow, and both are stated plainly rather than hidden. First, **AlphaGo-level or engine-level play is not an achievable target for this thesis.** Reaching it would require compute and budget of a different order, and claiming otherwise would be dishonest. Second, and more constructively, **this does not invalidate the research question.** The contribution is not "the strongest Xiangqi agent"; it is "a controlled comparison and an attribution of what limits each paradigm, measured rigorously at a stated budget." A result of the form "under budget B, paradigm X dominates paradigm Y, and the binding constraint is Z" is scientifically meaningful and is exactly what the literature currently lacks [15]. The budget is therefore reported as a parameter of the study rather than apologised for as a weakness.

## X. OUTPUT AND DEMONSTRABLE IMPACT

This section records the author's principal open concern and is the main subject on which guidance is sought.

### *A. The concern*

The present deliverables are a codebase, a set of Elo tables and a research report. These are appropriate academic outputs, but they are difficult for a non-specialist audience — including an examining committee — to *see*. The author is concerned that the work's value will not be visible from numbers alone, and wants an output that demonstrates the research rather than merely reporting it.

### *B. What already exists*

A working demonstrator has been built; Fig. 19 and Fig. 20 show it complete. Any two agents can be selected and played against each other, and an evaluation curve, computed by the engine, shows how the advantage shifts after every move. A full game is generated and replayed in a few seconds.




Fig. 19. The demonstrator, entry page. The complete page is shown.

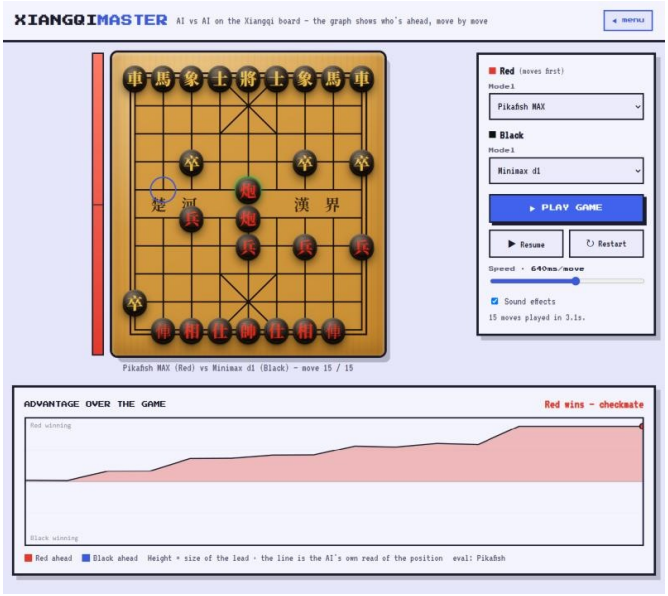



Fig. 20. The demonstrator, analysis page, shown complete from the header to the foot of the chart. Two agents are selected, the game is played out on the board, and the curve records the engine-computed advantage after every ply.

### *C. The proposed extension*

Following a recent discussion, the author proposes to extend the demonstrator into an interactive evaluation platform with three features.

1. **Human versus model play.** A visitor plays against any of the trained agents directly in the browser, which makes the difference between paradigms something the audience experiences rather than reads.
2. **A live rating.** Each human session contributes games, so each model carries a rating derived from both agent-versus-agent and human-versus-agent play, extending the league beyond self-contained experiments.
3. **Per-move explanation.** After each move the interface reports what the model was "thinking": the policy head's top candidate moves with their probabilities, the value head's assessment, and, for the search agent, the visit counts that determined the choice. Contrasting that with the engine's evaluation yields a natural judgement of whether a move was strong or weak, and *why* — for example, that the move dropped the evaluation by a large margin, or that the network's preferred move differed from the one search selected.

The scientific justification is that feature 3 converts the project from a benchmark into an *interpretability* instrument: it exposes the disagreement between a network's intuition and a search procedure's verification, which is the precise mechanism this thesis argues makes M4 the strongest paradigm. The engineering risk is scope creep, which is why this work is scheduled as a bounded block in Section XI and treated as a secondary objective that must not displace the scale-up study.

**Question for the supervisor.** Is this interactive platform an appropriate thesis output, or would the committee prefer

the effort to be concentrated entirely on the experimental study? The author has capacity for one of the two at the required depth, and would like guidance before committing.

## XI. PROJECT SCHEDULE

Table VII lists the work already completed and the work that remains, and Fig. 21 plots the same schedule against the calendar. Completed work is shown solid, planned work hatched, and the dashed line marks the present date.

TABLE VII
WORK COMPLETED AND WORK REMAINING

| Period | Work | Status |
|---|---|---|
| Sep 2026 | Rules engine, move generation, action space, tensor encoder, training environment | Complete |
| Sep–Oct 2026 | All four paradigms implemented over the shared substrate | Complete |
| Oct 2026 | Dataset D1 built and validated; imitation, PPO and MCTS training runs | Complete |
| Oct 2026 | Engine integration; dataset D2; distillation study | Complete |
| Oct 2026 | Evaluation league and the six studies; test-set accuracy report | Complete |
| Nov 2026 | Proposal, progress review, live demonstrator, this report | Complete |
| Nov–Dec 2026 | Supervisor consultation; freeze the experimental protocol | Planned |
| Dec 2026–Feb 2027 | Scale-up study: larger networks and larger self-play budgets, to test whether the ceiling of Section VIII is a budget artefact | Planned |
| Feb–Mar 2027 | Absolute-strength calibration against an engine ladder at several depths | Planned |
| Mar–Apr 2027 | Interactive platform, subject to the decision in Section X | Planned |
| Mar–May 2027 | Thesis writing | Planned |
| Apr–May 2027 | Revision, rehearsal, defense | Planned |
| End of May 2027 | **Thesis defense** | Milestone |

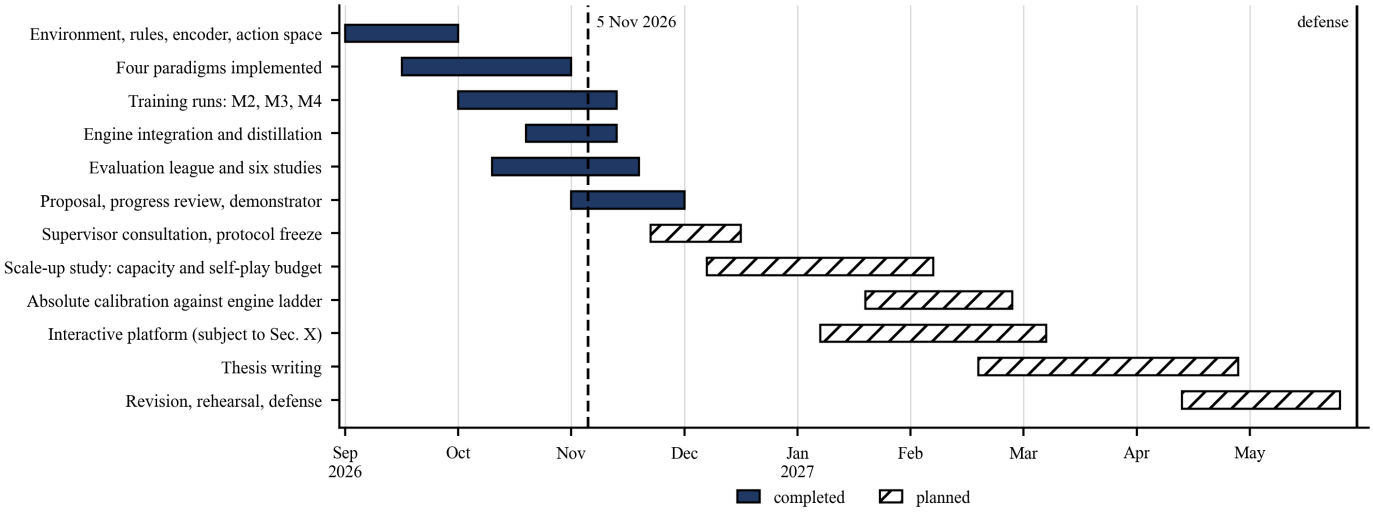



Fig. 21. Project schedule. All engineering and the first round of experiments are complete; the remaining work is scale-up, calibration, the output artefact, and writing.

## XII. EXPECTED CONTRIBUTIONS AND BENEFICIARIES

**Scientific.** The first controlled, like-for-like comparison of the four principal game-AI paradigms on Xiangqi, with an explicit attribution of what limits each one, addressing a methodological gap that has been argued for in the literature but rarely executed [15].

**Domain.** An addition to the small body of Xiangqi AI research [16], [19], [20], in a game whose complexity exceeds that of chess [16] but which remains comparatively unstudied.

**Practical.** Reusable artefacts: a validated rules engine, three documented datasets with a reproducible build, trained checkpoints, an evaluation league, and a demonstrator. A subsequent student can add a fifth paradigm by implementing a single interface.

**Educational.** If the interactive platform proceeds, a tool that lets a non-specialist see, move by move, how differently these four kinds of AI reason about the same position.

## XIII. RISKS AND MITIGATION

TABLE VIII
PRINCIPAL RISKS AND MITIGATION

| Risk | Likelihood | Impact | Mitigation |
|---|---|---|---|
| Scale-up study does not break the ceiling | Medium | Medium | A negative result is still publishable as a characterisation of the budget-limited regime; the report is framed so that either outcome answers RQ3 |
| Hardware limits or thermal throttling delay training | Medium | Medium | Checkpoint-and-resume throughout; cooled-burst scheduling; cloud GPU hours for the largest single run if required |
| Interactive platform consumes time needed for the study | Medium | High | Treated as a secondary objective with a fixed time box; decision deferred to the supervisor (Section X) |
| Elo intervals too wide to separate agents | Medium | Medium | Increase games per pair in the final league; report effect sizes and intervals rather than point ranks |
| Scope creep across four paradigms | Low | High | The shared-substrate design means no paradigm needs bespoke engineering |

## XIV. QUESTIONS FOR THE SUPERVISOR

The author would value the supervisor's judgement on the following, and is equally interested in criticism and in confirmation. Any of these decisions will be revised in line with the supervisor's advice.

1. **Output.** Is the interactive platform of Section X a suitable thesis output, or should the effort go entirely into the experimental study? If the platform is approved, how much of it is appropriate: human-versus-model play only, or also the per-move explanation and the live rating?
2. **Scope of the comparison.** Are four paradigms the right number, or would the committee prefer three studied more deeply?
3. **Framing of the compute limitation.** Is the position taken in Section IX-C — reporting the budget as a parameter of the study rather than as a shortcoming — the right way to present this to the committee?
4. **Absolute calibration.** Is calibration against an engine ladder sufficient, or should the work also attempt a mapping to human rating levels, which would require human games and raises its own validity questions?
5. **Publication.** Is any part of this of a standard worth preparing for submission, and if so, which part?
6. **Literature.** Are there works, particularly Chinese-language Xiangqi literature, that the author should read and that are not represented in Section III?

## REFERENCES

[1] C. E. Shannon, "Programming a computer for playing chess," *Philosophical Magazine*, vol. 41, no. 314, pp. 256–275, 1950.

[2] D. E. Knuth and R. W. Moore, "An analysis of alpha-beta pruning," *Artificial Intelligence*, vol. 6, no. 4, pp. 293–326, 1975.

[3] M. Campbell, A. J. Hoane Jr., and F.-h. Hsu, "Deep Blue," *Artificial Intelligence*, vol. 134, no. 1–2, pp. 57–83, 2002.

[4] D. A. Pomerleau, "Efficient training of artificial neural networks for autonomous navigation," *Neural Computation*, vol. 3, no. 1, pp. 88–97, 1991.

[5] A. Hussein, M. M. Gaber, E. Elyan, and C. Jayne, "Imitation learning: A survey of learning methods," *ACM Computing Surveys*, vol. 50, no. 2, art. 21, 2017.

[6] S. Ross, G. J. Gordon, and J. A. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in *Proc. 14th Int. Conf. Artificial Intelligence and Statistics (AISTATS)*, PMLR vol. 15, pp. 627–635, 2011.

[7] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," arXiv:1707.06347, 2017.

[8] O. Vinyals *et al.*, "Grandmaster level in StarCraft II using multi-agent reinforcement learning," *Nature*, vol. 575, pp. 350–354, 2019.

[9] D. Silver *et al.*, "Mastering the game of Go with deep neural networks and tree search," *Nature*, vol. 529, no. 7587, pp. 484–489, 2016.

[10] D. Silver *et al.*, "Mastering the game of Go without human knowledge," *Nature*, vol. 550, no. 7676, pp. 354–359, 2017.

[11] D. Silver *et al.*, "A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play," *Science*, vol. 362, no. 6419, pp. 1140–1144, 2018.

[12] R. Coulom, "Efficient selectivity and backup operators in Monte-Carlo tree search," in *Computers and Games*, LNCS vol. 4630, pp. 72–83, 2006.

[13] L. Kocsis and C. Szepesvári, "Bandit based Monte-Carlo planning," in *Proc. European Conf. Machine Learning (ECML)*, LNCS vol. 4212, pp. 282–293, 2006.

[14] C. B. Browne *et al.*, "A survey of Monte Carlo tree search methods," *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 4, no. 1, pp. 1–43, 2012.

[15] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger, "Deep reinforcement learning that matters," in *Proc. AAAI Conf. Artificial Intelligence*, vol. 32, no. 1, 2018.

[16] B. Yilmaz, J. Hu, and J. Liu, "Deep reinforcement learning Xiangqi player with Monte Carlo tree search," arXiv:2506.15880, 2025.

[17] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA: MIT Press, 2018.

[18] Y. Tian *et al.*, "ELF OpenGo: An analysis and open reimplementation of AlphaZero," in *Proc. 36th Int. Conf. Machine Learning (ICML)*, PMLR vol. 97, pp. 6244–6253, 2019.

[19] Y. Li *et al.*, "Mastering Xiangqi by tackling non-transitivity in two-player zero-sum games," *Transactions on Machine Learning Research (TMLR)*, 2023, arXiv:2308.04719.

[20] Y. Chen, S. Liu, Y. Lyu, C. Zhang, J. Shi, and T. Xu, "Xiangqi-R1: Enhancing spatial strategic reasoning in LLMs for Chinese chess via reinforcement learning," arXiv:2507.12215, 2025.

[21] J. Schrittwieser *et al.*, "Mastering Atari, Go, chess and shogi by planning with a learned model," *Nature*, vol. 588, pp. 604–609, 2020.

[22] A. E. Elo, *The Rating of Chessplayers, Past and Present*. New York: Arco, 1978.

[23] J. Hoffmann *et al.*, "Training compute-optimal large language models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[24] J. Kaplan *et al.*, "Scaling laws for neural language models," arXiv:2001.08361, 2020.

## APPENDIX A — PRESENTATION OUTLINE

For the oral presentation the following structure is planned, matching the required format.

1. **Introduction and overview** — what Xiangqi is, and the observation that four different kinds of AI have each claimed a landmark victory in games.
2. **Research question** — given one game and one budget, which paradigm wins, and what limits the others?
3. **Aim and objectives** — a controlled, reproducible comparison plus a reusable league (Section IV).
4. **Method and approach** — the shared substrate, the four paradigms, the three datasets, the evaluation protocol (Sections V–VII).
5. **Expected outcomes and summary** — the ranking obtained so far, the ceiling finding, the schedule to the defense, and the contribution to the field (Sections VIII, XI, XII).

The accompanying slide deck will use a dark background with high-contrast figures, and will reuse the diagrams in this report so that the visual language is consistent between the document and the talk.

## APPENDIX B — ANTICIPATED QUESTIONS

TABLE IX
ANTICIPATED QUESTIONS AND PREPARED RESPONSES

| Question | Prepared response |
|---|---|
| Why not just use AlphaZero, since it is the best? | It is included, as M4, and it does win. The research question is not which is best in the abstract but how the four compare under a stated budget, and what limits each — which the AlphaZero papers do not answer for Xiangqi. |
| Your agents are far weaker than Pikafish. Is the work still valuable? | Yes, because the contribution is the controlled comparison, not peak strength. The engine is included precisely so that the gap is measured and reported rather than ignored. |
| The network has only about one million parameters. Is that not very small? | It is small by design. This is a single-game policy-value network, not a general model; strength in board games comes from the combination of search and network, which is why the search agent built on this same small network is the strongest. The capacity ablation in Section VIII tested whether more parameters would help, and at this budget they did not. |
| Why is imitation learning beating self-play? Is that not the wrong way round? | At a large budget the literature expects self-play to win. The finding here is that at 500,000 steps it is not yet competitive, which is a statement about the budget, not a refutation of the method; this is stated explicitly in Section VIII-B. |
| Is the Elo comparable to a human rating? | No. It is a relative rating anchored within each tournament. Mapping to human ratings would require human games and is listed as an open question in Section XIV. |
| How do you know your data is clean? | Every game is replayed through the rules engine and discarded if any move is illegal; zero games failed, and an independent sample of five hundred replayed legally. |

If a question arises that the author cannot answer, the intended response is to acknowledge it honestly and undertake to follow it up — for example, *"That is an interesting point and I have not examined it yet; I will look into it and come back to you,"* or *"I do not have data on that, but it would be straightforward to test with the existing league, and I will add it."* Committing to a concrete follow-up is preferable to speculating.